

Physics Note: Pion Energy-Momentum Tensor Measurement

PyQUDA Meson EMT Workflow – Connected 3pt, Quark 1pt, and Gluon 1pt

PyQUDA Collaboration

July 16, 2026

Contents

1	Introduction and Physics Motivation	2
2	Code Architecture Overview	2
3	Part I: Connected Meson EMT	3
3.1	Meson Two-Point Function	3
3.2	Connected Three-Point Function (before sequential trick)	4
3.3	Scalar Diagnostic: C_3^χ	5
3.4	Connected Quark EMT Insertion	5
3.4.1	First term: derivative on the forward (source-to-insertion) line	5
3.4.2	Second term: derivative on the backward (sequential) line	5
3.4.3	Final contraction and symmetrization	6
3.5	Fixed-Sink Sequential Source Construction	7
3.6	Symmetric Covariant Derivative on a Propagator	8
3.7	Left Derivative on the Sequential Line	8
3.8	Gradient Flow	9
3.8.1	Flow schedule	9
3.8.2	Flowing propagators together	9
3.8.3	Gauge field flow	9
3.9	Connected 3pt High-Level Algorithm	9
3.10	HDF5 Output Format (Connected 3pt)	10
4	Part II: Disconnected One-Point Functions	10
4.1	Stochastic Quark One-Point Function	10
4.1.1	Noise generation	11
4.1.2	Identity channel and flowed-noise norm	11
4.1.3	Quark EMT one-point building block	11
4.1.4	Symmetrization, noise averaging, and volume normalization	12
4.1.5	HDF5 output format (Quark 1pt)	12
4.2	Ringed Fermion Normalization	12
4.3	Gluon One-Point Function	13
4.3.1	Clover field strength	13
4.3.2	Traceless projection	13
4.3.3	Gluon EMT building block	13
4.3.4	HDF5 output format (Gluon 1pt)	14
4.4	Gradient Flow Schedule (One-Point)	14
4.4.1	Full-volume source for a time-resolved flowed loop	14

4.5	Disconnected Matrix Element Analysis	15
4.5.1	Step 1 – Read input data per configuration	15
4.5.2	Step 2 – Time alignment: source position and absolute vs. relative time .	15
4.5.3	Step 3 – Kinematics and momentum matching	16
4.5.4	Step 4 – Form the disconnected three-point function	16
4.5.5	Step 5 – Ensemble averaging and vacuum subtraction	17
4.5.6	Step 6 – Ratio method and ground-state extraction	17
4.5.7	Step 7 – Assemble, renormalize, and decompose	18
4.6	Future Variance Reduction Strategies	19
4.7	Summary of All HDF5 Output Files	19
5	Technical Implementation Details	20
5.1	Parameter Dictionary	20
5.2	Inverter Parameters	20
5.3	Random Number Parameters (Quark 1pt)	20
5.4	Typical Usage Pattern	20
6	Cross-Checks and Consistency	21

1 Introduction and Physics Motivation

The energy-momentum tensor (EMT) $T_{\mu\nu}(x)$ is a fundamental operator in quantum field theory. In QCD it encodes the distribution of energy, momentum, pressure, and shear stress inside hadrons. Its matrix elements between hadron states,

$$\langle \pi(p_f) | T_{\mu\nu} | \pi(p_i) \rangle, \quad (1)$$

determine the gravitational form factors $A(q^2)$, $B(q^2)$, $D(q^2)$, and $\bar{c}(q^2)$ of the pion. These form factors provide access to the mass and spin decomposition of the pion, its mechanical radius, and the D -term (pressure distribution). Unlike the proton, the pion has spin zero, simplifying the Lorentz decomposition.

On the lattice, $T_{\mu\nu}$ requires a non-perturbative renormalization scheme. The gradient flow [1, 2] provides a regularization-independent method: flowing gauge and fermion fields to finite flow time t renders the bare composite operators finite. At small flow time, flowed operators are matched to $\overline{\text{MS}}$ via perturbatively computed matching coefficients $c_q(t)$ and $c_g(t)$:

$$T_{\mu\nu}^{\text{ren}}(x) = c_g(t) O_{\mu\nu}^g(t, x) + c_q(t) O_{\mu\nu}^q(t, x) + \text{mixing/trace terms}. \quad (2)$$

The mixing must account for the fact that the trace of $T_{\mu\nu}$ receives contributions from the trace anomaly. The matching coefficients $c_q(t)$, $c_g(t)$ are computed in perturbation theory and are part of the analysis stage, not the contraction kernel.

This note documents the complete numerical implementation in PyQUDA of:

- Connected meson three-point functions with quark EMT insertion (Section 3),
- Stochastic quark one-point functions for disconnected diagrams (Section 4.1),
- Gluon one-point functions (Section 4.3),

together with the gradient-flow schedule and HDF5 output formats. The target audience is lattice practitioners who need a line-by-line correspondence between the continuum formulas and the einsum contraction patterns in the Python source.

Key conventions

Throughout this note:

- Euclidean metric with γ_4 hermitian: $\gamma_\mu^\dagger = \gamma_\mu$, $\{\gamma_\mu, \gamma_\nu\} = 2\delta_{\mu\nu}$.
- $\gamma_5 = \gamma_1\gamma_2\gamma_3\gamma_4 = \gamma_5^\dagger$.
- Gamma hermiticity of the quark propagator: $S(x, y)^\dagger = \gamma_5 S(y, x) \gamma_5$.
- Convention B meson contraction with `meson_sign` = +1.
- Source position $x_0 = (t_0, \vec{x}_0)$, insertion $x = (\tau, \vec{x})$, sink $y = (t_{\text{sep}}, \vec{y})$.
- Momentum phase: $\Phi_k(r-x_0) = e^{ik \cdot (r-x_0)}$ where k uses continuum normalization $k_\mu \in \frac{2\pi}{L_\mu} \mathbb{Z}$.

2 Code Architecture Overview

The class hierarchy is:

<code>EMTDisconnectedQuark1pt</code>	shared quark-1pt loop
<code>EMTDisconnectedGluon1pt</code>	shared gluon-1pt loop
<code>QuarkEMT(EMTDisconnectedQuark1pt)</code>	connected meson 3pt + 2pt

`QuarkEMT` inherits all stochastic 1pt methods (including covariant derivatives, gamma5 hermiticity, and flow utilities) while adding meson-specific two-point and three-point contractions.

The application-level entry points are:

- `application/EMT_meson/perlmutter/Pyquda.EMT_quark.3pt.py` – connected 3pt
- `application/EMT_disconnected_1pt/perlmutter/` – shared quark/gluon loops

3 Part I: Connected Meson EMT

3.1 Meson Two-Point Function

The physical connected meson two-point function with source interpolator Γ_{src} and sink interpolator Γ_{sink} is

$$C_2^{\Gamma_{\text{sink}}}(p, t) = \sum_{\vec{x}} \Phi_{-p}(x - x_0) \text{Tr}_{sc} \left[S_{\text{anti}}(x, x_0) \Gamma_{\text{sink}} S_q(x, x_0) \Gamma_{\text{src}} \right], \quad (3)$$

where the antiquark line is built using gamma5 hermiticity:

$$S_{\text{anti}}(x, x_0) = \gamma_5 S_q(x, x_0)^\dagger \gamma_5. \quad (4)$$

In the code (`contract_meson_2pt`), the independent backward propagator `prop_bw` is used to form the antiquark line:

$$\text{bw_prop}(x) = \gamma_5 \text{prop_bw}(x, x_0)^\dagger \gamma_5, \quad (5)$$

and the contraction is computed as

$$C_2 = \sum_{\vec{x}} \Phi_{-p}(x - x_0) \text{Tr}_{sc} [\text{bw_prop}(x) \Gamma_{\text{sink}} \text{prop_fw}(x, x_0) \Gamma_{\text{src}}]. \quad (6)$$

The implementation scans all 16 gamma structures for the sink while keeping Γ_{src} fixed. The 16 gamma matrices (in `my_gammas`) are:

They are the shared DeGrand–Rossi matrices from `fermion_bilinear_basis.py`:

$$\gamma_1 = \begin{pmatrix} 0 & 0 & 0 & i \\ 0 & 0 & i & 0 \\ 0 & -i & 0 & 0 \\ -i & 0 & 0 & 0 \end{pmatrix}, \quad \gamma_2 = \begin{pmatrix} 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \end{pmatrix}, \quad (7)$$

$$\gamma_3 = \begin{pmatrix} 0 & 0 & i & 0 \\ 0 & 0 & 0 & -i \\ -i & 0 & 0 & 0 \\ 0 & i & 0 & 0 \end{pmatrix}, \quad \gamma_4 = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}, \quad (8)$$

with $\gamma_5 = \text{diag}(1, 1, -1, -1)$. The HDF5 file stores the same numerical matrices in `gamma_matrices`.

"5" = γ_5 ,	"T" = γ_4 ,
"T5" = $\text{raw} - \gamma_4 \gamma_5$,	"X" = γ_1 ,
"X5" = $\gamma_1 \gamma_5$,	"Y" = γ_2 ,
"Y5" = $\text{raw} - \gamma_2 \gamma_5$,	"Z" = γ_3 ,
"Z5" = $\gamma_3 \gamma_5$,	"I" = $\mathbf{1}$,
"SXT" = σ_{14} ,	"SXY" = σ_{12} ,
"SXZ" = σ_{13} ,	"SYT" = σ_{24} ,
"SYZ" = σ_{23} ,	"SZT" = σ_{34} ,

The raw PyQUDA tensor masks obey $S_{\mu\nu} = \frac{1}{2}[\gamma_\mu, \gamma_\nu]$ without an extra i ; multiply them by i if the Hermitian convention $\sigma_{\mu\nu} = \frac{i}{2}[\gamma_\mu, \gamma_\nu]$ is desired. Likewise the raw bit-mask labels satisfy $Y5_{\text{phys}} = -Y5_{\text{raw}}$ and $T5_{\text{phys}} = -T5_{\text{raw}}$ when every physical axial matrix is defined as $\gamma_\mu \gamma_5$. These transformations are stored explicitly in `physical_from_pyquda`. The ordering is fixed by `pyquda_gammas_order`: [15, 8, 7, 1, 14, 2, 13, 4, 11, 0, 9, 3, 5, 10, 6, 12].

In the einsum contraction, the data layout is

$$[\text{parity}, \text{t}, \text{z}, \text{y}, \text{x}, \text{spin_src}, \text{color_src}, \text{spin_sink}, \text{color_sink}]$$

so the einsum pattern for forming `bw_prop` is

```
bw_prop = contract("ij, wtzyxilab, kl -> wtzyxkjba",
                   G5, prop_bw.conj(), G5)
```

where index $i, j, k, l \in \{0, 1, 2, 3\}$ (spin) and $a, b \in \{0, 1, 2\}$ (color). The transpose of spin-sink and spin-source indices (`ilab` $\rightarrow$ `kjba`) implements the dagger with spin-color reordering.

To avoid a 16-fold propagator-sized intermediate, the sink Gamma insertion and scalar folding are performed one channel at a time:

```
for gamma_idx, sink_gamma in enumerate(sink_gammas):
    sink_inserted = contract("wtzyxjicf, im->wtzyxjmcf",
                             bw_prop, sink_gamma)
    scalar = contract("wtzyxjiab, wtzyxilba, lj->wtzyx",
                     sink_inserted, prop_fw, src_gamma)
    C2_local[gamma_idx] = contract("qwtzyx, wtzyx->qt", phases, scalar)
```

Thus the persistent result still has all 16 sink channels, but no temporary array has both a Gamma axis and all propagator spin-color indices.

Boosted smearing: If `CG_GaussSmear` is enabled, Gaussian smearing with a momentum boost is applied at both source and sink:

- Source smearing uses boost `pos_boost` on the forward source and `neg_boost` on the backward source (when they differ).
- Sink smearing is applied during the 2pt contraction with the same `pos_boost` and `neg_boost`.

The smearing operator used throughout this note is the FFT boosted Gaussian operator $\mathcal{S}_{w, \mathbf{k}}$. Its real-space kernel is

$$K_{\mathbf{k}}(\mathbf{r}) = \exp\left(-\frac{r_x^2 + r_y^2 + r_z^2}{2w^2}\right) \exp\left[2\pi i \left(\frac{k_x r_x}{L_x} + \frac{k_y r_y}{L_y} + \frac{k_z r_z}{L_z}\right)\right], \quad (9)$$

where $\mathbf{k}$ is the integer vector supplied by `pos_boost` or `neg_boost`. The current `boosted_smearing_pyquda.py` implementation applies a three-dimensional spatial FFT convolution,

$$\psi_{\mathbf{k}}^{\text{smear}}(\mathbf{x}, t) = \mathcal{F}_{3d}^{-1} [\mathcal{F}_{3d}[\psi](\mathbf{p}, t) \mathcal{F}_{3d}[K_{\mathbf{k}}](\mathbf{p})] (\mathbf{x}, t), \quad (10)$$

to every spin-color source column of a propagator. The FFT is spatial only and the same kernel is used on each Euclidean time slice. No explicit gauge rotation is applied in the current implementation.

3.2 Connected Three-Point Function (before sequential trick)

For a local quark bilinear insertion $\mathcal{O}_g(x)$ at the space-time point $x = (\tau, \vec{x})$ and a final sink with momentum p_f , the connected three-point function is

$$C_3^g(q, \tau; p_f, t_{\text{sep}}) = \sum_{\vec{x}} \sum_{\vec{y}} \Phi_q(x-x_0) \Phi_{p_f}(y-x_0) \text{Tr}_{sc} \left[S_{\text{anti}}(y, x_0) \Gamma_{\text{sink}} S_q(y, x) \mathcal{O}_g(x) S_q(x, x_0) \Gamma_{\text{src}} \right]. \quad (11)$$

Here q is the momentum transfer (denoted `qext` in the code), p_f is the fixed sink momentum (`pf`), and t_{sep} is the source-sink separation. The notation follows the usual three-point pattern: the quark propagates from source x_0 to insertion x , the operator is inserted, the quark continues to sink y , and the antiquark runs directly from source to sink.

3.3 Scalar Diagnostic: C_3^χ

With $\mathcal{O}_g(x) = \mathbf{1}$ (the unit operator in spin-color space), the three-point function reduces to

$$C_3^\chi(q, \tau) = \sum_{\vec{x}} \Phi_q(x - x_0) \text{Tr}_{sc} \left[S_{\text{seq-anti}}(x) S_q(x, x_0) \Gamma_{\text{src}} \right], \quad (12)$$

where $S_{\text{seq-anti}}(x)$ is the backward sequential meson line defined in Section 3.5. This diagnostic verifies that the sequential source construction is correct: a well-constructed sequential source should give C_3^χ that is numerically consistent with the two-point function when the scalar insertion is trivial.

In `get_C3_chi`, the contraction is

```
dst2 = make_dst2(seq_bw_prop) # gamma5 * S_seq^dag * gamma5
scalar_field = contract("wtzyxabij, wtzyxbcji, ca -> wtzyx",
                        dst2, prop_fw.data, src_gamma)
# Fourier transform to momentum space, keep time
slice_t = gatherLattice( contract("qwtzyx, wtzyx -> qt",
                                phases_3pt, scalar_field) )
```

Note the spin indices: `dst2` has layout `(..., a=spin_sink, b=spin_src, i=color_sink, j=color_src)`, while `prop_fw.data` has `(..., b=spin_src, c=spin_sink, j=color_src, i=color_sink)`. The contraction `abij, bcji, ca` traces over spin and color, with `src_gamma` providing the source interpolator.

3.4 Connected Quark EMT Insertion

The Euclidean connected quark EMT insertion is

$$T_{\mu\nu}^q(x) = \frac{1}{2} \left[\gamma_\nu D_\mu - \overleftarrow{D}_\mu \gamma_\nu \right], \quad (13)$$

where D_μ is the symmetric covariant derivative acting to the right and $\overleftarrow{D}_\mu$ acts to the left. This bilinear is not Hermitian by itself; the symmetrization $\mu \leftrightarrow \nu$ is performed after contraction to obtain a Hermitian EMT.

When inserted into the three-point function, two distinct contributions arise depending on which quark line the derivative acts upon:

3.4.1 First term: derivative on the forward (source-to-insertion) line

$$C_3^{\mu\nu, q, \text{first}}(q, \tau) = +\frac{1}{2} \sum_{\vec{x}} \Phi_q(x - x_0) \text{Tr}_{sc} \left[S_{\text{seq-anti}}(x) \gamma_\nu (D_\mu S_q(x, x_0)) \Gamma_{\text{src}} \right]. \quad (14)$$

The derivative acts on the forward propagator *before* multiplication by the gamma matrix. The overall sign is $+1/2$.

3.4.2 Second term: derivative on the backward (sequential) line

$$C_3^{\mu\nu, q, \text{second}}(q, \tau) = -\frac{1}{2} \sum_{\vec{x}} \Phi_q(x - x_0) \text{Tr}_{sc} \left[(\overleftarrow{D}_\mu S_{\text{seq-anti}}(x)) \gamma_\nu S_q(x, x_0) \Gamma_{\text{src}} \right]. \quad (15)$$

The derivative acts on the sequential backward propagator *from the left*. The overall sign is $-1/2$, which follows from the minus sign in Eq. (13) for the left-derivative term.

3.4.3 Final contraction and symmetrization

The full quark EMT three-point function is the sum of the two contributions:

$$C_3^{\mu\nu,q}(q,\tau) = C_3^{\mu\nu,q,\text{first}}(q,\tau) + C_3^{\mu\nu,q,\text{second}}(q,\tau). \quad (16)$$

After this sum, explicit symmetrization enforces $T_{\mu\nu} = T_{\nu\mu}$:

$$C_3^{\mu\nu,q}(q,\tau) \rightarrow \frac{1}{2}(C_3^{\mu\nu,q}(q,\tau) + C_3^{\nu\mu,q}(q,\tau)). \quad (17)$$

Channel by channel, the batched primitive kernel implements this as:

```
# First term: +1/2 * Tr[dst2 * gamma_nu * (D_mu prop_fw) * Gamma_src]
for mu in range(4):
    D_fw = covdev_sym_prop(U_f, prop_fw, mu)
    for nu in range(4):
        gamma_D_fw = contract("ab, wtzyxbdi j -> wtzyxadi j",
                               D_gammas[nu], D_fw.data)
        scalar = 0.5 * contract("wtzyxabij, wtzyxbcji, ca -> wtzyx",
                                dst2, gamma_D_fw, src_gamma)

# Second term: -1/2 * Tr[(leftD_mu dst2) * gamma_nu * prop_fw * Gamma_src]
for mu in range(4):
    leftD_dst2 = left_covdev_dst2_from_prop(U_f, seq_bw_prop, mu)
    for nu in range(4):
        gamma_fw = contract("ab, wtzyxbdi j -> wtzyxadi j",
                             D_gammas[nu], prop_fw.data)
        scalar = -0.5 * contract("wtzyxabij, wtzyxbcji, ca -> wtzyx",
                                leftD_dst2, gamma_fw, src_gamma)

# Symmetrize
for mu in range(4):
    for nu in range(mu+1, 4):
        C3_Tmunu[:, mu, nu] = 0.5 * (C3_Tmunu[:, mu, nu]
                                       + C3_Tmunu[:, nu, mu])
        C3_Tmunu[:, nu, mu] = C3_Tmunu[:, mu, nu]
```

The production contraction now evaluates all 16 gamma matrices for each of the four derivative directions. The primitive array is unsymmetrized; selecting the vector channels $\gamma_1, \gamma_2, \gamma_3, \gamma_4$ and symmetrizing reproduces the previous `C3.Tmunu` exactly. Here derivative index zero is x , one is y , two is z , and three is Euclidean time. The batched Gamma contraction adds no propagator inversion, fermion-flow step, or covariant derivative call.

Remark 1 (Trace ordering) *The einsum pattern `abij, bcji, ca` should be read as:*

- *a: spin-sink of `dst2` (or `leftD_dst2`)*
- *b: spin-source of `dst2`, spin-sink of forward propagator*
- *c: spin-source of forward propagator, spin index of `src_gamma`*
- *i, j: color-sink and color-source, traced by Einstein summation*
- *ca: `src_gamma` with indices (spin-source, spin-source-of-gamma)*

The color trace is implicit in the repeated i and j indices.

3.5 Fixed-Sink Sequential Source Construction

The fixed-sink method absorbs the sum over the sink position y into a sequential propagator. This avoids an $O(V^2)$ summation and makes the three-point calculation tractable.

Step 1: Build the source-smear, sink-smear forward propagator.

Starting from the source-smear point-sink forward propagator `prop_fw.SP(y, x0)`, optional sink smearing is applied:

$$\text{prop_fw_SS}(y, x_0) = \text{boosted_smearing}(\text{prop_fw.SP}(\cdot, x_0), \text{boost} = \text{pos_boost})(y). \quad (18)$$

Step 2: Restrict to the sink time slice and apply momentum phase.

The function `create_meson_bw_seq_pyquda` (in `bw_seq_pyquda.py`) constructs the sequential source:

$$\eta_{\text{seq}}(y; p_f, t_{\text{sep}}) = \delta_{t_y, t_0 + t_{\text{sep}}} \Phi_{p_f}(y - x_0) \Gamma_{\text{seq}} \text{prop_fw_SS}(y, x_0), \quad (19)$$

where the time-slice restriction is implemented via `pyquda.utils.source.sequential12`, and

$$\Gamma_{\text{seq}} = \gamma_5 \Gamma_{\text{sink}}^\dagger \gamma_5. \quad (20)$$

In the einsum implementation:

```
gamma_seq = einsum("ab, cb, cd -> ad", G5, sink_gamma.conj(), G5)
seq_data = einsum("wtzyx, wtzyxlab -> wtzyxlab", mom_phase, seq_data)
seq_data = einsum("ij, wtzyxlab -> wtzyxlab", gamma_seq, seq_data)
```

Step 2b: Smear the active sequential source.

For a smeared-smeared connected three-point function the code smears the sequential source before inversion:

$$\eta_{\text{seq}}^{SS} = \mathcal{S}_{w, \text{neg_boost}} [\delta_{t_y, t_0 + t_{\text{sep}}} \Phi_{p_f}(y - x_0) \Gamma_{\text{seq}} \text{prop_fw_SS}(y, x_0)]. \quad (21)$$

The input `prop_fw_SS` has already received the `pos_boost` sink smearing. The outer $\mathcal{S}_{w, \text{neg_boost}}$ represents the active line associated with the backward sequential propagator in the current pion EMT implementation.

Step 3: Invert the Dirac operator.

$$D S_{\text{seq}} = \eta_{\text{seq}}. \quad (22)$$

Step 4: Form the backward sequential meson line.

Using `gamma5` hermiticity, the line that connects the insertion to the sink is

$$S_{\text{seq_anti}}(x; p_f, t_{\text{sep}}) = \gamma_5 S_{\text{seq}}(x)^\dagger \gamma_5. \quad (23)$$

This is computed in `_make_dst2`:

```
dst2 = contract("ab, wtzyxbcij, cd -> wtzyxadij",
                G5, prop.data.conj().transpose(0,1,2,3,4,6,5,8,7), G5)
```

The `transpose(0,1,2,3,4,6,5,8,7)` swaps spin-source/sink (indices 5 $\leftrightarrow$ 6) and color-source/sink (indices 7 $\leftrightarrow$ 8), implementing the Hermitian conjugate with the correct index layout.

Step 5: Post-sequential three-point function.

After the sequential inversion, the sink-summed three-point function takes the compact form:

$$C_3^g(q, \tau; p_f, t_{\text{sep}}) = \sum_{\vec{x}} \Phi_q(x - x_0) \text{Tr}_{sc} \left[S_{\text{seq_anti}}(x; p_f, t_{\text{sep}}) \mathcal{O}_g(x) S_q(x, x_0) \Gamma_{\text{src}} \right]. \quad (24)$$

This is the master formula used by `get_C3_primitive_bilinears`; the identity local channel gives `C3_chi`, while the four vector derivative channels give `C3.Tmunu`.

3.6 Symmetric Covariant Derivative on a Propagator

The symmetric (improved) covariant derivative acting on a quark propagator is defined on the lattice as

$$D_{\mu}^{\text{sym}} S(x) = \frac{1}{2} \left[U_{\mu}(x) S(x + \hat{\mu}) - U_{-\mu}(x) S(x - \hat{\mu}) \right], \quad (25)$$

where $U_{-\mu}(x) = U_{\mu}^{\dagger}(x - \hat{\mu})$. The forward direction $+\mu$ corresponds to direction index $\mu \in \{0, 1, 2, 3\}$, and the backward direction $-\mu$ corresponds to $\mu + 4 \in \{4, 5, 6, 7\}$ in the QUDA direction enumeration.

The code (`_covdev_sym_prop`) applies this spin-color component by component via `MultiFermion`:

```
mf = propagatorToMultiFermion(prop)
for spin in range(4):
    for color in range(3):
        idx = spin * 3 + color
        Dp = gauge_dirac.covDev(mf[idx], mu)      # +mu
        Dm = gauge_dirac.covDev(mf[idx], mu + 4)  # -mu
        mf_covdev[idx] = 0.5 * (Dp - Dm)
```

Here `gauge_dirac` is owned by the caller. Production enters one `with U.f.use()` context per output flow time and reuses that same resident flowed gauge for both quark legs and all eight forward/backward derivative directions. This changes only gauge loading: the derivative order, signs, Gamma contractions, and saved arrays are unchanged.

The inverter still performs one full gauge load when it constructs the initial multigrid hierarchy. The first source and first fixed-sink sequential inversions use that already-resident state directly. Inversions after a flowed-gauge context restore the same original thin gauge with `thin_update_only=True`; near-null vectors and the coarse hierarchy are not rebuilt. The connected workflow remains one branch at a time and does not batch different separations or sources.

The one-branch flow still carries 24 full fermion fields. In the cfg1050 light-quark 64^4 smoke with 16 ranks (local volume 32^4), this existing flow exhausted an 80-GB A100 even after the unrelated C2 contraction was omitted. A production setup at that local volume therefore needs a finer MPI decomposition or a separate connected-memory optimization; thin restore does not reduce the 24-field flow allocation.

Remark 2 (Sign convention) *The minus sign between D_+ and D_- in the code, $0.5 \times (D_+ - D_-)$, corresponds to the standard lattice symmetric derivative where `covDev` implements the forward/backward parallel transport without the link division by $2a$. The factor $1/2$ in the formula provides the continuum normalization.*

3.7 Left Derivative on the Sequential Line

The second term of the quark EMT insertion requires the left-acting derivative on the backward sequential line $S_{\text{seq_anti}}$. Rather than implementing a separate left-derivative routine, the code exploits gamma5 hermiticity:

$$\overleftarrow{D}_{\mu} S_{\text{seq_anti}}(x) = \gamma_5 (D_{\mu} S_{\text{seq}}(x))^{\dagger} \gamma_5. \quad (26)$$

The derivation is:

$$\overleftarrow{D}_{\mu} S_{\text{seq_anti}} = \overleftarrow{D}_{\mu} (\gamma_5 S_{\text{seq}}^{\dagger} \gamma_5) \quad (27)$$

$$= \gamma_5 (D_{\mu} S_{\text{seq}})^{\dagger} \gamma_5, \quad (28)$$

where the derivative passes through the constant gamma matrices and acts on $S_{\text{seq}}^\dagger$ from the right, which is equivalent to the Hermitian conjugate of the right-acting derivative.

The implementation in `_left_covdev_dst2_from_prop` is:

```
D_y = covdev_sym_prop(U_f, prop, mu)    # D_mu S_seq
D_y_dag = D_y.data.conj().transpose(0,1,2,3,4,6,5,8,7)  # (D_mu S_seq)^dag
leftD_dst2 = contract("ab, wtzyxbcij, cd -> wtzyxadij",
                      G5, D_y_dag, G5)
```

The resulting `leftD_dst2` encodes $\overleftarrow{D}_\mu S_{\text{seq-anti}}(x)$ and is used directly in the second-term contraction of Eq. (15).

3.8 Gradient Flow

Gradient flow is applied to smooth UV fluctuations. The flow radius is approximately $\sqrt{8t}$ where t is the flow time. The code supports both Wilson flow and Symanzik (Lüscher-Weisz) flow via the `flow_type` parameter.

3.8.1 Flow schedule

The flow schedule follows a **measure-before-flow** convention:

1. **Step 0:** Measure observables on the unflowed fields. Output index `step=0` is unflowed.
2. **First interval (step 0 → 1):** Advance fields by 10 small sub-steps of size $\varepsilon/10$.
3. **Subsequent intervals:** Advance by one step of size ε .

Thus output index `step` corresponds to flow time $t = \text{step} \times \varepsilon$.

3.8.2 Flowing propagators together

The forward propagator and sequential backward propagator are flowed simultaneously as a single `MultiField` on the flowed gauge background:

```
mf_a = propagatorToMultiFermion(prop_a)
mf_b = propagatorToMultiFermion(prop_b)
packed = multiField(mf_a_all + mf_b_all)
packed_flow = U_f.gradientFlow(packed, flow_type, Nsteps, stepsize)
```

This ensures both fermion fields see the same flowed gauge links at every step.

3.8.3 Gauge field flow

The gauge field itself is flowed by `U_f.wilsonFlow()` or `U_f.symanzikFlow()`, which modifies the gauge links in place. Before flowing, anti-periodic boundary conditions in time are set via `U_f.setAntiPeriodicT()`.

3.9 Connected 3pt High-Level Algorithm

The `connected_3pt` method orchestrates the full measurement:

1. Build source-smeared point-sink forward and backward propagators (`_make_meson_source_props`).
2. Contract the meson 2pt function as a diagnostic (`contract_meson_2pt`).
3. For each source-sink separation t_{sep} :
 - (a) Build and invert the meson fixed-sink sequential source (`create_meson_bw_seq_pyquda`).
 - (b) For each flow step:
 - i. Compute all 16 local and 16×4 derivative channels once via `get_C3_primitive_bilinears`.

- ii. Derive C_3^χ from the identity channel and $C_3^{\mu\nu,q}$ from the symmetrized vector derivative channels.
 - iii. Advance the flowed propagators via `_advance_floweds_props`.
4. Save the results to HDF5.

3.10 HDF5 Output Format (Connected 3pt)

The output file (written by `save_emt_quark_3pt_hdf5`) contains:

- `C3_chi`: 4D array `[N_ts, Nsteps+1, Nq, Nt]` – scalar diagnostic.
- `C3_Tmunu`: 6D array `[N_ts, Nsteps+1, Nq, 4, 4, Nt]` – quark EMT 3pt.
- `C3_local_bilinear`: `[N_ts, 16, Nq, Nsteps+1, Nt]`.
- `C3_derivative_bilinear`: `[N_ts, 16, 4, Nq, Nsteps+1, Nt]`.
- `gamma_list`, `gamma_pyquda_ids`, `gamma_matrices`, `physical_from_pyquda`, and `derivative_directions`: exact operator-basis metadata.
- `momentum_transfer_list`: list of `[q_x, q_y, q_z, q_t]` arrays.
- HDF5 attributes: `measurement`, `flow_type`, `flow_epsilon`, `flow_steps`, `n_t_separations`, `src_interpolator`, `sink_interpolator`, `contraction_convention` (“B”), `meson_sign` (+1), `n_qext`. The spin-zero pion channel is specified only by the source and sink interpolators; no separate spin-projection label is used.

The connected primitive axes differ from the disconnected EMTc axes. The following reconstruction reproduces the stored tensor exactly:

```
with h5py.File(path, "r") as h5:
    labels = [x.decode() for x in h5["gamma_list"][...]]
    vector = [labels.index(x) for x in ("X", "Y", "Z", "T")]
    D = h5["C3_derivative_bilinear"][...]
    # D: [tsep, gamma, derivative, q, flow, t]
    B = np.take(D, vector, axis=1)
    T = 0.5 * (B + np.swapaxes(B, 1, 2))
    T_file = T.transpose(0, 4, 3, 1, 2, 5)
    np.testing.assert_allclose(T_file, h5["C3_Tmunu"][...])
```

No spatial-volume factor is applied here: these are already contracted connected correlators. The exact raw matrices and physical-basis conversion are documented in `docs/EMT_gamma_and_raw_bilinears.md`.

The four main connected arrays have logical storage weights

$$\text{C3_derivative}:\text{C3_local}:\text{C3_Tmunu}:\text{C3_chi} = 64 : 16 : 16 : 1. \quad (29)$$

Thus in a large file their shares approach 65.98%, 16.49%, 16.49%, and 1.03%, respectively. Unlike the disconnected canonical file, the connected `C3_Tmunu` is a sizable derived copy retained for direct downstream use.

The meson 2pt diagnostic is saved separately by `save_emt_meson_2pt_hdf5` with:

- `C2`: 3D array `[16, N_mom, Nt]` – 16 sink gammas $\times$ all momenta $\times$ time.
- `gamma_list`: list of gamma labels.
- `momentum_list`: list of momentum vectors.

4 Part II: Disconnected One-Point Functions

4.1 Stochastic Quark One-Point Function

The disconnected quark loop requires the trace of the quark propagator with operator insertions. These are estimated stochastically using $\mathbb{Z}_n$ noise vectors.

4.1.1 Noise generation

A $\mathbb{Z}_n$ noise fermion $\xi(x)$ is generated with random complex phases:

$$\xi(x) \in \{e^{2\pi i r/n} \mid r \in \{0, 1, \dots, n-1\}\}, \quad (30)$$

with `n = n_zn`; the production default is $n = 4$. Each phase is constructed by the fixed counter described in `docs/EMT_disconnected_1pt/EMT_disconnected_1pt.tex`, using the global coordinate, spin, color, configuration, base-noise index, and stream salt. Thus the global source is unchanged by the MPI decomposition and obeys $\langle \xi(x) \rangle = 0$ and $\langle \xi(x) \xi^\dagger(y) \rangle = \delta_{xy}$ in the noise ensemble.

The solution vector is obtained by inverting the Dirac operator:

$$\eta(x) = D^{-1} \xi(x). \quad (31)$$

4.1.2 Identity channel and flowed-noise norm

Two scalar quantities monitor the stochastic estimator:

$$\text{CHI}[0](q, t) = \sum_{\vec{x}} \Phi_q(x) \xi^\dagger(x) \eta(x) \quad (32)$$

$$\text{CHI}[1](q, t) = \sum_{\vec{x}} \Phi_q(x) \xi^\dagger(x) \xi(x) \quad (33)$$

The first quantity is the identity channel of the complete 16-Gamma local basis and is not duplicated. The second is the flowed-noise norm.

In code:

```
scalar_trace = local_bilinear[gamma_list.index("I")]

dot_xi_xi = contract("etzyxbc, etzyxbc -> etzyx",
                     xi.data.conj(), xi.data)
flowed_noise_norm = impose_P_Breit_slice(dot_xi_xi, phases_3pt)
```

4.1.3 Quark EMT one-point building block

The stochastic estimate of the quark EMT one-point function is

$$T_{\nu\mu}^q(q, t) = -\frac{1}{2} \sum_{\vec{x}} \Phi_q(x) \xi^\dagger(x) \gamma_\nu [D_{+\mu} - D_{-\mu}] \eta(x). \quad (34)$$

Here $D_{\pm\mu}$ are the forward/backward covariant derivatives (NOT the symmetrized versions – the difference $D_+ - D_-$ already implements the symmetric derivative up to a factor of 2). The indices on $T_{\nu\mu}^q$ are ordered (ν, μ) because the gamma matrix index comes first in the code:

```
for mu in range(4):
    tmp = covDev(eta, mu) - covDev(eta, mu + 4) # (D_+mu - D_-mu) eta
    for nu in range(4):
        Y = contract("ab, ...bc -> ...ac", D_gammas[nu], tmp.data)
        complex_field = contract("...sc, ...sc -> ...",
                                  xi.data.conj(), Y)
        Tmunu[nu, mu] += -0.5 * impose_P_Breit_slice(...)
```

The minus sign $(-1/2)$ follows from Eq. (13) combined with the stochastic trace convention.

4.1.4 Symmetrization, noise averaging, and volume normalization

After accumulating all μ, ν components, the tensor is symmetrized (identical to the connected case):

$$T_{\mu\nu}^q \rightarrow \frac{1}{2}(T_{\mu\nu}^q + T_{\nu\mu}^q). \quad (35)$$

The measurements are repeated for `n_vec` independent noise vectors and averaged:

$$\langle T_{\mu\nu}^q \rangle = \frac{1}{N_{\text{vec}}} \sum_{v=1}^{N_{\text{vec}}} T_{\mu\nu}^{q,(v)}. \quad (36)$$

Finally, the result is divided by the spatial volume $V_3 = L_x L_y L_z$ to obtain the intensive one-point density:

$$T_{\mu\nu}^{q,\text{avg}}(q, t, t_f) = \frac{1}{V_3} \langle T_{\mu\nu}^q(q, t, t_f) \rangle. \quad (37)$$

4.1.5 HDF5 output format (Quark 1pt)

The canonical EMTc (written by the shard finalizer) contains:

- `raw/local_bilinear_pervec`: [`n_vec`, 16, `Nq`, `Nsteps+1`, `Nt`].
- `raw/derivative_bilinear_pervec`: [`n_vec`, 16, 4, `Nq`, `Nsteps+1`, `Nt`].
- `raw/flowed_noise_norm_pervec`: [`n_vec`, `Nq`, `Nsteps+1`, `Nt`].
- `avg/local_bilinear` and `avg/derivative_bilinear`: source-averaged, volume-normalized primitives.
- `avg/Tmunu/T11`, `T12`, ..., `T44`: averaged, volume-normalized tensor components (symmetric storage: only $\mu \leq \nu$).
- `avg/flowed_noise_norm`: [`Nq`, `Nsteps+1`, `Nt`].
- Attributes: `measurement`, `flow_type`, `flow_epsilon`, `flow_steps`, `n_vec`, `n_zn`.

4.2 Ringed Fermion Normalization

The ringed fermion renormalization scheme [3] requires the flowed fermion kinetic expectation value. At zero momentum, the raw diagonal quark EMT one-point gives, for each effective stochastic source r ,

$$K_r(t_f, \tau) = -\frac{2}{V_s} \sum_{\mu=1}^4 T_{\mu\mu,r}^q(0, t_f, \tau) \quad (38)$$

$$= \frac{1}{V_s} \sum_{\mu, \vec{x}} \xi^\dagger(x, t_f) \gamma_\mu [D_{+\mu} - D_{-\mu}] \eta(x, t_f). \quad (39)$$

The EMT 1pt run extracts this identity directly from the vector diagonal channels of `raw/derivative_bilinear_pervec`, without another solve, flow update, derivative, or gather, and writes `derived/ringed/kinetic_pervec` and `derived/ringed/kinetic_spacetime` in the same EMTc; it does not contain configuration-local `Z_ring` datasets. The same quantity can be checked against

$$\text{avg/Tmunu/T11, T22, T33, T44}$$

at the $q = 0$ momentum index. Final ringed factors must be evaluated from the ensemble average of `derived/ringed/kinetic_spacetime`; averaging factors formed as the inverse of each configuration's kinetic value is not equivalent. The local identity and flowed-noise norm

remain diagnostics. The complete convention and schema are documented in the disconnected one-point note. Production one-point jobs write base/HP-interval shards by default. Only the explicit finalizer publishes the canonical EMTc with embedded kinetic data after all configured bases have complete HP coverage.

4.3 Gluon One-Point Function

4.3.1 Clover field strength

The gauge field strength $F_{\mu\nu}(x)$ is constructed via the clover (plaquette + rectangle) operator. The standard clover loop for the (μ, ν) plane combines four oriented paths:

$$\begin{aligned} \text{loop}_1 &: \mu \rightarrow \nu \rightarrow -\mu \rightarrow -\nu, \\ \text{loop}_2 &: \nu \rightarrow -\mu \rightarrow -\nu \rightarrow \mu, \\ \text{loop}_3 &: -\mu \rightarrow -\nu \rightarrow \mu \rightarrow \nu, \\ \text{loop}_4 &: -\nu \rightarrow \mu \rightarrow \nu \rightarrow -\mu. \end{aligned} \tag{40}$$

The field strength is extracted as

$$F_{\mu\nu}(x) = \frac{1}{8} \left(Q_{\mu\nu}(x) - Q_{\mu\nu}^\dagger(x) \right), \tag{41}$$

where $Q_{\mu\nu}$ is the sum of the four clover loops (each with unit coefficient). The resulting $F_{\mu\nu}$ is anti-Hermitian by construction.

4.3.2 Traceless projection

The field strength is projected onto the traceless anti-Hermitian $\mathfrak{su}(3)$ algebra:

$$F_{\mu\nu}^{\text{su}(3)}(x) = F_{\mu\nu}(x) - \frac{1}{N_c} \text{Tr}_c [F_{\mu\nu}(x)] \cdot \mathbf{1}_{N_c}. \tag{42}$$

The code then multiplies by $-i$ to match the Euclidean convention:

$$F_{\mu\nu}^{\text{Eucl}}(x) = -i F_{\mu\nu}^{\text{su}(3)}(x). \tag{43}$$

In `_F_clover_traceless`:

```
A = 0.125 * (data - data.swapaxes(-2, -1).conjugate())
trA = contract("...ii -> ...", A)
A -= trA[..., None, None] * I / Nc
data[...] = (-1j) * A
```

4.3.3 Gluon EMT building block

The gluonic EMT one-point function is

$$T_{\mu\nu}^g(q, t) = \frac{2}{V_3} \sum_{\vec{x}} \Phi_q(x) \sum_{\substack{\rho=0 \\ \rho \neq \mu, \nu}}^3 \text{Tr}_c [F_{\mu\rho}(x) F_{\nu\rho}(x)]. \tag{44}$$

The sum over ρ runs over all Lorentz indices except μ and ν . This is the full gluonic EMT building block (not a traceless projection of the EMT itself – the traceless projection in `_F_clover_traceless` operates only on the field-strength level to ensure $\mathfrak{su}(3)$ algebra).

In code:

```

for mu in range(4):
    for nu in range(mu, 4):
        tmp = zeros(...)
        for rho in range(4):
            if rho == mu or rho == nu: continue
            tmp += contract("...ab, ...ba -> ...", F[mu][rho], F[nu][rho])
        Tmunu_t[mu, nu, :, step, :] += 2.0 * slice_t

```

The factor 2 is explicit in the formula, and the volume normalization $1/V_3$ occurs after the spatial Fourier sum.

Antisymmetry: $F_{\mu\nu}$ is antisymmetric by construction, so only the six independent planes $(0,1), (0,2), (0,3), (1,2), (1,3), (2,3)$ are computed, and $F_{\nu\mu} = -F_{\mu\nu}$ is filled by antisymmetry.

4.3.4 HDF5 output format (Gluon 1pt)

The file (written by `save_empt_gluon_1pt_hdf5`) contains:

- Tmunu/T11, T12, ..., T44: 3D arrays [Nq, Nsteps+1, Nt] – volume-normalized tensor components (symmetric storage: only $\mu \leq \nu$).
- Attributes: `measurement`, `flow_type`, `flow_epsilon`, `flow_steps`.

4.4 Gradient Flow Schedule (One-Point)

The one-point flow schedule mirrors the connected 3pt schedule (measure-before-flow, step-0 is unflowed). There is one important difference for the quark 1pt: the noise vector ξ and solution η are flowed together as a single `MultiField`. This ensures that the flowed ξ and η correspond to the same flowed gauge background.

Quark 1pt flow:

```

for step in range(Nsteps + 1):
    measure primitive bilinears, flowed_noise_norm
    if Nsteps > 0 and step == 0:
        temp = multiField([xi, eta])
        temp_flow = U_f.gradientFlow(temp, flow_type, 10, stepsize/10)
        xi, eta = temp_flow[0], temp_flow[1]
    elif Nsteps > 0 and step < Nsteps:
        temp = multiField([xi, eta])
        temp_flow = U_f.gradientFlow(temp, flow_type, 1, stepsize)
        xi, eta = temp_flow[0], temp_flow[1]

```

Gluon 1pt flow:

```

for step in range(Nsteps + 1):
    measure Tmunu
    if Nsteps > 0 and step == 0:
        U_flow.wilsonFlow(10, epsilon=stepsize/10) # or symanzik
    elif Nsteps > 0 and step < Nsteps:
        U_flow.wilsonFlow(1, epsilon=stepsize)      # or symanzik

```

4.4.1 Full-volume source for a time-resolved flowed loop

The quark one-point estimator retains the absolute insertion time even though its initial stochastic source is four dimensional. Let $K(t_f)$ be the gauge-covariant fermion-flow kernel and P_τ the projector onto the absolute insertion time τ . The flowed stochastic fields and estimator are

$$\xi_f = K(t_f)\xi, \quad \eta_f = K(t_f)D^{-1}\xi, \quad \hat{L}(\tau, t_f) = \xi^\dagger K^\dagger P_\tau \Gamma K D^{-1} \xi. \quad (45)$$

Consequently,

$$\mathbb{E}_\xi[\widehat{L}(\tau, t_f)] = \text{Tr} \left[P_\tau \Gamma K(t_f) D^{-1} K^\dagger(t_f) \right]. \quad (46)$$

The projector keeps a spatial trace at fixed τ ; it does not sum over insertion time. Nevertheless, fermion flow uses the four-dimensional covariant Laplacian and spreads in time with characteristic radius approximately $\sqrt{8t_f}$. Restricting the initial source to one time projector generally omits finite-flow contributions. A complete time-dilution basis would remain unbiased only after summing every projector, with the corresponding inversion cost. The current workflow instead uses full-volume counter-based Z_4 noise and stores all τ . The complete derivation is given in `docs/EMT_disconnected_1pt/EMT_disconnected_1pt.tex`.

Spatial and four-dimensional hierarchical probing do not change the source support. Both multiply a full-volume base noise; spatial HP only makes the probing sign independent of time.

4.5 Disconnected Matrix Element Analysis

The contraction kernel outputs *bare flowed operators* to HDF5 files (Sec. 4.7). This section documents the full analysis chain that turns those outputs into physical gravitational form factors. The procedure is organized in six steps.

4.5.1 Step 1 – Read input data per configuration

For each gauge configuration, four HDF5 files are read:

File	Dataset	Shape
EMT2pt/...h5	C2 gamma_list momentum_list	[16, Np, Nt] 16 strings [Np, 4]
EMT3pt/...h5	C2 C3_chi C3_Tmunu	[Nt] (zero-momentum, one [Nts, Ns+1, Nq, Nt] [Nts, Ns+1, Nq, 4, 4, N
EMTc/...h5	avg/Tmunu/Tij avg/flowed_noise_norm raw/local_bilinear_pervec raw/derivative_bilinear_pervec	[Nq, Ns+1, Nt] each [Nq, Ns+1, Nt] [Nv, 16, Nq, Ns+1, Nt] [Nv, 16, 4, Nq, Ns+1, Nt]
EMTg/<lat>.EMTg.<cfg>.<ama>.<sm>.h5	Tmunu/Tij	[Nq, Ns+1, Nt] each

Notation: Np = number of 2pt momenta, Nq = number of momentum transfers, Nts = number of t_{sep} values, Ns = number of flow steps, Nv = number of noise vectors, Nt = temporal lattice extent.

For the disconnected analysis, the relevant interpolator is $\Gamma = \gamma_5$ (pseudoscalar, labeled "5" in `gamma_list`). The sink-gamma index `ig5` is located by matching against `gamma_list`. The momentum index `ip` for a desired sink momentum p_f is located by matching against `momentum_list`.

4.5.2 Step 2 – Time alignment: source position and absolute vs. relative time

The meson 2pt function C_2 is computed with a specific source located at $x_0 = (t_0, \vec{x}_0)$. The source time t_0 is encoded in the file tag (e.g., ...x0y0z0t32...). The 2pt dataset C2 has shape [16, Np, Nt] where the last axis is the **absolute sink time** $t_{\text{sink}} \in [0, N_t - 1]$.

The 1pt loops $L_{\mu\nu}(q, \tau)$ also use **absolute lattice time** $\tau \in [0, N_t - 1]$ for the operator insertion. The 1pt measurement is source independent: its canonical EMTc file omits the `src`

label and one full-time loop file per configuration is reused for every two-point source time. The value of t_0 must therefore be read from the corresponding hadron two-point file, not from the EMTc file.

The physically meaningful time coordinates are *relative to the source*:

$$t_{\text{sep}} = (t_{\text{sink}} - t_0) \bmod N_t, \quad \tau_{\text{rel}} = (\tau_{\text{abs}} - t_0) \bmod N_t. \quad (47)$$

In practice the source is placed away from the temporal boundary and $t_{\text{sep}} \ll N_t$, so the modulo operation is a formality. The analysis loops over t_{sep} values of interest and, for each, extracts C_2 at $t_{\text{sink}} = t_0 + t_{\text{sep}}$ and L at $\tau_{\text{abs}} = t_0 + \tau_{\text{rel}}$ (with trivial wrap-around handling).

Location of t_0 in the HDF5 files. For source-dependent hadron correlators, the source position x_0 is stored in:

- The file name (e.g., ...x0y0z0t32... – the t component is t_0).
- The HDF5 attribute `src_interpolator` or `source_position` (convention varies by file type; the hadron-correlator file name is the authoritative source).

The canonical quark loop name is EMTc/<lat>.EMTc.<cfg>.<ama>.<sm>.h5 and intentionally contains no source position.

Parsing example (Python pseudocode):

```
import re
fname = "lat.EMT2pt.cfg100.ama0.x0y0z0t32.sm0.h5"
t0 = int(re.search(r"t(?:\d+)", fname).group(1))
```

4.5.3 Step 3 – Kinematics and momentum matching

The meson 2pt and the 1pt loops have independent momentum lists. The source pion carries momentum p_i , the sink carries p_f (from `momentum_list` in the 2pt file), and the operator insertion carries momentum q (from `qext`). Momentum conservation at the operator vertex requires

$$q = p_f - p_i. \quad (48)$$

The source momentum p_i is determined by the source construction:

- For a point source without boost: $p_i = (0, \vec{0})$.
- For a Gaussian-smeared source with boost `pos_boost`: $p_i = \text{pos_boost}$ (up to smearing-induced smearing of the momentum distribution).

Thus, for each pair ($p_f \in \text{p_2pt}$, $q \in \text{qext}$) satisfying $q = p_f - p_i$, the relevant 2pt and 1pt entries are matched. The **simplest and most common case** is $p_i = 0$, for which $p_f = q$ and the momentum lists can be chosen identical (`p\_2pt = qext`).

In the analysis loop, for each momentum transfer q :

1. Compute $p_f = q + p_i$.
2. Find the index `ip` of p_f in `momentum_list`.
3. Find the index `iq` of q in `qext` (from HDF5 attributes or the `momentum_transfer_list` dataset).
4. The 2pt at this kinematics is `C2[ig5, ip, :]`.
5. The 1pt loop at this kinematics is `Tmunu[iq, :, :]`.

4.5.4 Step 4 – Form the disconnected three-point function

For a single gauge configuration, fixed flow step `is` (flow time $t_f = \text{is} \times \varepsilon$), fixed Lorentz indices (μ, ν) , and a matched momentum pair (p_f, q) , the disconnected three-point function at relative insertion time τ and source-sink separation t_{sep} is

$$C_3^{\text{disc}, q}(q, \tau, t_{\text{sep}}, t_f) \big|_{\text{cfg}} = C_2(p_f, t_{\text{sink}}=t_0+t_{\text{sep}}) \times L_{\mu\nu}^q(q, \tau_{\text{abs}}=t_0+\tau, t_f). \quad (49)$$

Indices on the right-hand side refer to the stored array axes:

- `C2[ig5, ip, t0 + tsep] - 2pt` at sink gamma γ_5 , sink momentum p_f , absolute sink time $t_0 + t_{\text{sep}}$.
 - `L_quark[iq, istep, t0 + tau]` – quark loop at momentum transfer q , flow step `is`, absolute insertion time $t_0 + \tau$.
 - `L_gluon[iq, istep, t0 + tau]` – gluon loop, same indexing.
- The gluon loop enters identically to the quark loop:

$$C_3^{\text{disc},g}(q, \tau, t_{\text{sep}}, t_f)|_{\text{cfg}} = C_2(p_f, t_0 + t_{\text{sep}}) \times L_{\mu\nu}^g(q, t_0 + \tau, t_f). \quad (50)$$

Only the window $0 < \tau < t_{\text{sep}}$ contributes to the ground-state signal (the operator must be inserted between source and sink). Points outside this window serve as cross-checks (should be consistent with zero after vacuum subtraction).

The connected three-point function C_3^{conn} is read directly from `EMT3pt/...h5` (dataset `C3.Tmunu`, shape `[Nts, Ns+1, Nq, 4, 4, Nt]`). It already includes the sequential-source summation and uses the same source x_0 ; its time axis is absolute and must be aligned identically.

4.5.5 Step 5 – Ensemble averaging and vacuum subtraction

Let N_{cfg} be the number of gauge configurations. For each jackknife or bootstrap resampling bin b , compute the ensemble averages:

$$\langle C_2(p, t_{\text{sep}}) \rangle_b = \frac{1}{N_{\text{cfg}} - 1} \sum_{\text{cfg} \neq b} C_2(p, t_{\text{sep}})|_{\text{cfg}}, \quad (51)$$

$$\langle L_{\mu\nu}(q, \tau, t_f) \rangle_b = \frac{1}{N_{\text{cfg}} - 1} \sum_{\text{cfg} \neq b} L_{\mu\nu}(q, \tau, t_f)|_{\text{cfg}}, \quad (52)$$

$$\langle C_2 \cdot L_{\mu\nu} \rangle_b = \frac{1}{N_{\text{cfg}} - 1} \sum_{\text{cfg} \neq b} [C_2(p, t_{\text{sep}}) L_{\mu\nu}(q, \tau, t_f)]_{\text{cfg}}. \quad (53)$$

The disconnected three-point function with vacuum subtraction is then

$$\boxed{C_3^{\text{disc}}(q, \tau, t_{\text{sep}}, t_f) = \langle C_2 \cdot L \rangle - \langle C_2 \rangle \langle L \rangle}. \quad (54)$$

This subtraction removes the vacuum expectation value of the operator and the disconnected contribution where the pion propagates without interacting with the EMT. Eq. (54) is applied separately to the quark loop (L^q) and gluon loop (L^g).

The jackknife error on C_3^{disc} is computed in the standard way: $\sigma^2 = \frac{N_{\text{cfg}} - 1}{N_{\text{cfg}}} \sum_b (C_3^{\text{disc}}(b) - \overline{C_3^{\text{disc}}})^2$.

4.5.6 Step 6 – Ratio method and ground-state extraction

The bare matrix element is isolated by forming the ratio with the two-point function:

$$R_{\mu\nu}^{\mathcal{O}}(q, \tau, t_{\text{sep}}, t_f) = \frac{C_3^{\mathcal{O}}(q, \tau, t_{\text{sep}}, t_f)}{\langle C_2(p_f, t_{\text{sep}}) \rangle}, \quad (55)$$

where $\mathcal{O} \in \{\text{conn}, \text{disc-}q, \text{disc-}g\}$. For the connected piece, C_3^{conn} from the 3pt file enters the numerator. For the disconnected pieces, C_3^{disc} from Eq. (54) enters.

In the limit of large source-insertion and insertion-sink separations, excited states decay exponentially and the ratio plateaus to the ground-state matrix element:

$$R_{\mu\nu}^{\mathcal{O}}(q, \tau, t_{\text{sep}}, t_f) \xrightarrow{\tau \gg 0, t_{\text{sep}} - \tau \gg 0} \frac{\langle \pi(p_f) | T_{\mu\nu}^{\mathcal{O}}(t_f) | \pi(p_i) \rangle}{2\sqrt{E_\pi(p_i) E_\pi(p_f)}}. \quad (56)$$

With $p_i = 0$ and $E_\pi(0) = m_\pi$, this simplifies to

$$R_{\mu\nu}^{\mathcal{O}} \xrightarrow{\text{plateau}} \frac{\langle \pi(p_f) | T_{\mu\nu}^{\mathcal{O}}(t_f) | \pi(0) \rangle}{2\sqrt{m_\pi E_\pi(p_f)}}. \quad (57)$$

Plateau fitting procedure:

1. For each t_{sep} , plot $R(\tau)$ as a function of τ .
2. Identify the plateau region where $R(\tau)$ is flat within errors. Typical choices: $\tau \in [2, t_{\text{sep}} - 2]$ or $\tau \in [t_{\text{sep}}/3, 2t_{\text{sep}}/3]$.
3. Fit $R(\tau)$ to a constant (or constant + single-exponential correction) in the plateau window.
4. The fit result is the bare matrix element $M_{\mu\nu}^{\mathcal{O}, \text{bare}}(q, t_f, t_{\text{sep}})$.
5. As a cross-check, verify that the extracted M is independent of t_{sep} (within errors) for sufficiently large t_{sep} .

Alternatively, the **summation method** integrates over τ :

$$S_{\mu\nu}^{\mathcal{O}}(q, t_{\text{sep}}, t_f) = \sum_{\tau=\tau_{\text{min}}}^{t_{\text{sep}}-\tau_{\text{min}}} R_{\mu\nu}^{\mathcal{O}}(q, \tau, t_{\text{sep}}, t_f), \quad (58)$$

whose slope against t_{sep} gives the matrix element with reduced excited-state contamination at the cost of larger statistical errors.

4.5.7 Step 7 – Assemble, renormalize, and decompose

Total bare matrix element. After ground-state extraction, the three pieces are summed at each flow time t_f :

$$M_{\mu\nu}^{\text{bare}}(q, t_f) = M_{\mu\nu}^{\text{conn}}(q, t_f) + M_{\mu\nu}^{\text{disc}, q}(q, t_f) + M_{\mu\nu}^{\text{disc}, g}(q, t_f). \quad (59)$$

All three pieces share the same lattice kinematics and flow schedule, so they can be summed directly.

Renormalization and flow-time dependence. The bare flowed matrix elements are converted to the $\overline{\text{MS}}$ scheme at scale μ using perturbative matching coefficients [7, 3]:

$$M_{\mu\nu}^{\overline{\text{MS}}}(q, \mu) = \lim_{t_f \rightarrow 0} \left[c_q(t_f, \mu) M_{\mu\nu}^{q, \text{bare}}(q, t_f) + c_g(t_f, \mu) M_{\mu\nu}^{g, \text{bare}}(q, t_f) + \delta_{\mu\nu} c_{\text{trace}}(t_f, \mu) \sum_{\rho, \sigma} M_{\rho\sigma}^{\text{bare}}(q, t_f) \right]. \quad (60)$$

Here $M_{\mu\nu}^{q, \text{bare}} = M_{\mu\nu}^{\text{conn}} + M_{\mu\nu}^{\text{disc}, q}$ is the total quark contribution, and $M_{\mu\nu}^{g, \text{bare}} = M_{\mu\nu}^{\text{disc}, g}$ is the total gluon contribution. The trace term accounts for the trace anomaly; c_{trace} is determined by the renormalization condition $\sum_\mu T_{\mu\mu}^{\text{ren}} = (\beta/2g) F_{\rho\sigma}^a F_{\rho\sigma}^a + (1 + \gamma_m) m \bar{\psi}\psi$.

In practice, the $t_f \rightarrow 0$ limit is implemented by:

1. Extracting the matrix element at each flow time $t_f = \text{step} \times \varepsilon$.
2. Applying the matching coefficients $c_q(t_f, \mu)$, $c_g(t_f, \mu)$ at each t_f .
3. Performing a linear (or constant) extrapolation in t_f to $t_f = 0$, using only the small- t_f window where perturbation theory is reliable (typically $\sqrt{8t_f} \lesssim 0.3 \text{ fm}$).

Gravitational form factor decomposition. For a spin-0 pion, the Lorentz decomposition of the renormalized EMT matrix element is [5]:

$$\langle \pi(p_f) | T_{\mu\nu}^{\overline{\text{MS}}}(\mu) | \pi(p_i) \rangle = 2 P_\mu P_\nu A(q^2) + \frac{1}{2} (q_\mu q_\nu - \delta_{\mu\nu} q^2) D(q^2) + 2 m_\pi^2 \delta_{\mu\nu} \bar{c}(q^2), \quad (61)$$

where $P = (p_i + p_f)/2$ and $q = p_f - p_i$. The three gravitational form factors are:

- $A(q^2)$ – the mass form factor ($A(0) = 1$ for a stable hadron),
- $D(q^2)$ – the D -term encoding pressure and shear distributions,
- $\bar{c}(q^2)$ – the trace-anomaly form factor ($\bar{c}(0)$ gives the gluon contribution to the pion mass).

Given $M_{\mu\nu}(q) \equiv \langle \pi(p_f) | T_{\mu\nu} | \pi(p_i) \rangle$ from the lattice, the form factors are extracted by solving the 4×4 linear system at each q^2 . With $p_i = 0$, the components simplify:

$$M_{44}(q) = 2E_\pi^2(q) A(q^2) + \frac{1}{2}(q_4^2 + \vec{q}^2) D(q^2) + 2m_\pi^2 \bar{c}(q^2), \quad (62)$$

$$M_{ii}(q) = 2q_i^2 A(q^2) + \frac{1}{2}(q_i^2 - \vec{q}^2) D(q^2) + 2m_\pi^2 \bar{c}(q^2) \quad (i = 1, 2, 3), \quad (63)$$

$$M_{4i}(q) = 2E_\pi(q) q_i A(q^2) + \frac{1}{2} q_4 q_i D(q^2), \quad (64)$$

$$M_{ij}(q) = 2q_i q_j A(q^2) + \frac{1}{2} q_i q_j D(q^2) \quad (i \neq j). \quad (65)$$

In practice, $A(q^2)$, $D(q^2)$, and $\bar{c}(q^2)$ are overdetermined (10 independent $M_{\mu\nu}$ components for 3 unknowns at each q^2) and are extracted by a least-squares fit to Eqs. (62)–(65).

Summary of the full analysis chain.

1. **Read** HDF5 files for all N_{cfg} configurations.
2. **Match** momenta: $p_f = q$, locate indices in `momentum_list` and `qext`.
3. **Form** $C_3^{\text{disc}, q}$ and $C_3^{\text{disc}, g}$ per configuration via Eq. (49).
4. **Jackknife** over configurations; compute $\langle C_2 \rangle$, $\langle L \rangle$, $\langle C_2 L \rangle$; subtract vacuum via Eq. (54).
5. **Ratio** $R = C_3 / \langle C_2 \rangle$; plateau-fit in τ for each $(q, t_f, \mu, \nu, t_{\text{sep}})$.
6. **Sum** connected + disconnected quark + gluon $\rightarrow M_{\mu\nu}^{\text{bare}}(q, t_f)$.
7. **Renormalize** with $c_q(t_f, \mu)$, $c_g(t_f, \mu)$; extrapolate $t_f \rightarrow 0$.
8. **Solve** for $A(q^2)$, $D(q^2)$, $\bar{c}(q^2)$ via Eqs. (62)–(65).

4.6 Future Variance Reduction Strategies

The current implementation uses plain $\mathbb{Z}_n$ noise for the stochastic quark 1pt estimator. Two natural upgrades are identified for future production:

1. **Hierarchical probing** [4] (arXiv:1302.4018): The EMT 1pt trace estimator is $\text{Tr}(\Gamma D^{-1})$ with local gamma and derivative insertions. Hierarchical probing replaces or supplements random noise vectors with distance-ordered Hadamard/coloring vectors on the 4D torus. This cancels near-diagonal contributions of D^{-1} more deterministically, reducing stochastic variance at fixed inversion count. The method is particularly effective for operators with local support like $T_{\mu\nu}$.
2. **Frequency splitting / propagator decomposition** (arXiv:2605.00643): The flowed EMT loop can be decomposed into low/infrared and high/ultraviolet components using frequency-filtered estimator pieces. This allows expensive noise averaging to be focused on the component with the largest residual variance. The strategy should be designed together with the gradient-flow radius since flow already smooths ultraviolet modes.

When implemented, these upgrades should preserve the existing HDF5 schema or add explicit estimator metadata so that disconnected diagram analysis can combine old and new 1pt data safely.

4.7 Summary of All HDF5 Output Files

Notation: N_p = number of momenta, N_q = number of momentum transfers, N_{ts} = number of t_{sep} values, N_s = number of flow steps, N_v = number of noise vectors, N_t = temporal lattice extent.

Table 1: Summary of HDF5 output files produced by the pion EMT workflow.

File type	Tag subdirectory	Main datasets	Shape
Meson 2pt	EMT2pt/	C2	[16, Np, Nt]
Quark 3pt	EMT3pt/	C2	[Nt]
		C3_chi	[Nts, Ns+1, ...]
		C3_Tmunu	[Nts, Ns+1, ...]
Quark 1pt	EMTc/	raw/derivative_bilinear_pervec	[Nv, 16, 4, Nq, ...]
		avg/Tmunu/Tij	[Nq, Ns+1, Nt]
Gluon 1pt	EMTg/<lat>.EMTg.<cfg>.<ama>.<sm>.h5	Tmunu/Tij	[Nq, Ns+1, Nt]

5 Technical Implementation Details

5.1 Parameter Dictionary

The `QuarkEMT` class is initialized with a parameter dictionary containing:

Key	Description
<code>qext</code>	list of momentum transfers $[q_x, q_y, q_z, q_t]$
<code>pf</code>	fixed sink momentum $[p_x, p_y, p_z, p_t]$
<code>p_2pt</code>	list of momenta for the 2pt function
<code>CG_GaussSmear</code>	whether to apply Gaussian smearing
<code>pos_boost</code>	momentum boost for forward source/sink
<code>neg_boost</code>	momentum boost for backward source/sink
<code>width</code>	Gaussian smearing width
<code>flow_type</code>	"wilson" or "symanzik"
<code>flow_epsilon</code>	flow step size ϵ
<code>flow_steps</code>	number of flow steps

5.2 Inverter Parameters

The Dirac operator uses the QUDA multigrid solver with parameters:

```
invPara = [mass, csw, tol, maxiter]
```

Typical values are `mass` = 0.236, `csw` = 1.0372, `tol` = 1e-10, and `maxiter` = 300. Connected pion and proton entypoints share `--mg-block X.Y.Z.T[;...]`, defaulting to one level 8.8.4.4; `--mg-block none` disables multigrid. The actual hierarchy is stored in both C2 and C3 attributes.

5.3 Random Number Parameters (Quark 1pt)

`randPara` = [`n_vec`, `n_zn`, `randseed`] where `n_vec` is the number of base noise vectors, `n_zn` is the $\mathbb{Z}_n$ order (default 4), and `randseed` is the counter stream salt (default 0). The gauge configuration number is a separate counter key. No rank-local backend RNG is used.

5.4 Typical Usage Pattern

The application scripts follow a common pattern:

1. **Initialize QUDA:** `init(mpi_geometry, enable_mps=True)`
2. **Read gauge field:** `io.readNERSCGauge(gauge_path)`
3. **Apply HYP smearing:** `gauge.hypSmear(1, 0.75, 0.6, 0.3, 4)`

4. **Set boundary condition:** `gauge.latt_info.t_boundary = -1` (anti-periodic in time)
5. **Instantiate measurement class:** `quark_empt = QuarkEMT(parameters)`
6. **Run measurement:** `quark_empt.connected_3pt(...)`
7. **Results saved to HDF5** automatically.

6 Cross-Checks and Consistency

- **Convention B + meson_sign = +1:** All meson 2pt and 3pt contractions are internally consistent with this choice. Zero-momentum regression tests match previous baselines to roundoff precision.
- **CHI diagnostic:** C_3^X with $\mathcal{O}_g = \mathbf{1}$ must be numerically consistent with the meson 2pt function (up to a sink momentum phase factor). This is a strong check of the sequential source construction.
- **Symmetrization:** Both connected and disconnected EMT tensors are symmetrized $\mu \leftrightarrow \nu$ after the full contraction. The antisymmetric part should be zero within statistics.
- **Flow schedule alignment:** The quark and gluon flow schedules are aligned (measure-before-flow, first interval subdivided). Output index `step` corresponds to flow time $t = \text{step} \times \varepsilon$ in all three file types.
- **Volume normalization:** The connected 3pt output is NOT volume-normalized (it is an extensive correlator). The 1pt outputs ARE divided by V_3 (intensive densities). This distinction is critical for the analysis stage when forming $\langle C_2 L \rangle$.

References

- [1] M. Lüscher, “Properties and uses of the Wilson flow in lattice QCD,” JHEP **08** (2010), 071. [arXiv:1006.4518 [hep-lat]].
- [2] M. Lüscher and P. Weisz, “Perturbative analysis of the gradient flow in non-abelian gauge theories,” JHEP **02** (2011), 051. [arXiv:1101.0963 [hep-th]].
- [3] H. Makino and H. Suzuki, “Lattice energy-momentum tensor from the Yang-Mills gradient flow – inclusion of fermion fields,” PTEP **2014** (2014), 063B02. [arXiv:1404.2758 [hep-lat]].
- [4] A. Stathopoulos, J. Laeuchli, and K. Orginos, “Hierarchical probing for estimating the trace of the matrix inverse on toroidal lattices,” [arXiv:1302.4018 [hep-lat]].
- [5] L. Del Debbio, A. Patella, and C. Pica, “Energy-momentum tensor on the lattice: recent developments,” [arXiv:2105.08278 [hep-lat]].
- [6] T. Harris, G. von Hippel, P. Junnarkar, H. B. Meyer, K. Ottnad, J. Wilhelm, H. Wittig, and L. Wrang, “Nucleon isovector tensor form factors from the lattice,” Phys. Rev. D **100** (2019), 034513. [arXiv:1905.01291 [hep-lat]].
- [7] H. Suzuki, “Energy-momentum tensor from the Yang-Mills gradient flow,” PTEP **2013** (2013), 083B03. [arXiv:1304.0533 [hep-lat]].